SETUP

실습 환경 구성

모든 세션의 실습은 Python 트랙과 Java 트랙으로 제공합니다. 개인 PC에 한 번 설치해 두면 24세션을 그대로 따라 할 수 있고, 사내 폐쇄망으로 옮기는 방법도 함께 정리했습니다.

한눈에 보기

Python과 Java 중 하나만 골라도 됩니다. 둘 다 설치하면 같은 실습을 두 언어로 비교해 볼 수 있습니다. 버전은 2026년 11월 기준입니다.

구분	도구	버전	필요한 세션
필수	Git	최신	전체
Python 트랙	Python	3.13 권장 (3.11 이상)	전체
Java 트랙	JDK (Eclipse Temurin)	21 LTS	전체
Java 트랙	Maven	프로젝트의 Maven Wrapper(`mvnw`)가 내려받음	전체
편집기	VS Code 또는 IntelliJ IDEA · PyCharm	최신	전체
선택	Docker (또는 Rancher Desktop · Podman)	최신	5·7(n8n), 8(pgvector), 11·12(Oracle Free), 16, 21
선택	Node.js	24 LTS	13(코딩 에이전트 CLI), 14~16(Next.js 트랙), 18(MCP Inspector)
선택	Ollama	최신	2·4(로컬 모델 비교), 21(폐쇄망 LLM)
선택	NVIDIA GPU	16~24GB 이상	21(vLLM), 22(파인튜닝)

Python 3.14 도 나와 있지만 vLLM 같은 일부 패키지는 지원이 늦습니다. 실습은 3.13 으로 맞추는 것을 권합니다.

공통 도구

Git · 편집기 · Node.js · Ollama

Windows

```
PowerShell

winget install --id Git.Git -e
winget install --id Microsoft.VisualStudioCode -e
winget install --id OpenJS.NodeJS.LTS -e     # 세션 13~16, 18
winget install --id Ollama.Ollama -e         # 세션 2, 4, 21
```

macOS

```
터미널 (zsh)

# Homebrew 가 없으면 https://brew.sh 의 설치 명령을 먼저 실행합니다
brew install git node@24
brew install --cask visual-studio-code
brew install --cask ollama                  # 세션 2, 4, 21
```

- winget 은 Windows 10·11 의 기본 패키지 관리자입니다. 회사 PC에서 막혀 있으면 각 공식 사이트의 설치 파일을 씁니다.
- IntelliJ IDEA 는 2025.3 부터 Community·Ultimate 가 하나로 합쳐졌고, 체험 기간이 끝나도 Java·Kotlin 핵심 기능은 무료입니다. JetBrains 안내

Docker

Docker Desktop 은 직원 250명 초과 또는 연매출 1,000만 달러 초과 기업에서 업무용으로 쓰려면 유료 구독이 필요합니다. 회사 PC라면 사내 정책을 먼저 확인하고, 필요하면 Rancher Desktop 이나 Podman Desktop 을 씁니다. 개인 PC 학습용은 무료입니다.

Python 트랙

1. Python 설치

Windows

```
PowerShell
```

```
winget install --id Python.Python.3.13 -e
# 설치 후 새 PowerShell 창에서
py -3.13 --version
```

python.org 설치 파일을 쓸 때는 첫 화면의 **Add python.exe to PATH** 를 꼭 체크합니다.

macOS

```
터미널 (zsh)

brew install python@3.13
python3.13 --version
```

2. 세션마다 가상환경 만들기

세션 폴더마다 `.venv` 를 만들어 패키지 버전이 서로 섞이지 않게 합니다.

Windows

```
PowerShell

cd labs\s06
py -3.13 -m venv .venv
.venv\Scripts\Activate.ps1          # 막히면 아래 '문제 해결'의 실행 정책 참고
pip install -r requirements.txt
python first_call.py
```

macOS

```
터미널 (zsh)

cd labs/s06
python3.13 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python first_call.py
```

파일 이름은 세션마다 다릅니다. 실행 명령은 각 세션 '실습 코드' 탭의 Python 트랙에 있습니다.

3. 편집기

- VS Code: 확장 **Python**(Microsoft) 설치 → 명령 팔레트 `Python: Select Interpreter` 에서 `.venv` 선택
- PyCharm: 폴더 열기 → 인터프리터를 `.venv` 로 지정

Java 트랙

1. JDK 21 설치 (Eclipse Temurin)

Windows

```
PowerShell

winget install --id EclipseAdoptium.Temurin.21.JDK -e
# 새 PowerShell 창에서
java -version
echo $env:JAVA_HOME
```

설치 파일(.msi)을 쓸 때는 **Set JAVA_HOME variable** 항목을 켭니다.

macOS

```
터미널 (zsh)

brew install --cask temurin@21
java -version
# JAVA_HOME 지정 (~/.zshrc 에 추가)
echo 'export JAVA_HOME=$(/usr/libexec/java_home -v 21)' >> ~/.zshrc
source ~/.zshrc
```

2. Maven Wrapper 로 빌드·실행

Java 실습은 모두 Maven 프로젝트입니다. 전역 Maven 을 설치하지 않아도 프로젝트에 들어 있는 Maven Wrapper(`mvnw`)가 맞는 Maven 을 내려받습니다.

Windows

```
PowerShell

cd labs\s06\java
.\mvnw.cmd -v
.\mvnw.cmd -q compile exec:java "-Dexec.mainClass=study.s06.FirstCall"
```

macOS

```
터미널 (zsh)

cd labs/s06/java
./mvnw -v
```

```
./mvnw -q compile exec:java -Dexec.mainClass=study.s06.FirstCall
```

모든 Java 실습 폴더에 `mvnw` · `mvnw.cmd` 가 들어 있습니다 (Maven Wrapper 3.3.4, Maven 3.9.16). 처음 실행할 때 Maven 을 `~/.m2/wrapper` 에 한 번 내려받고, 이후에는 그대로 씁니다.

3. 편집기

- IntelliJ IDEA: `pom.xml` 이 있는 폴더를 열면 Maven 프로젝트로 인식합니다. Project SDK 를 Temurin 21 로 지정합니다.
- VS Code: 확장 **Extension Pack for Java**(Microsoft) 설치. Spring 세션(14~16, 18)은 **Spring Boot Extension Pack** 도 설치합니다.

API 키와 환경변수

실습 코드는 키를 코드에 넣지 않고 환경변수에서 읽습니다. 실습용 개인 키를 쓰고, 사내 적용은 승인된 모델·게이트웨이 주소로 바꿉니다.

Windows

PowerShell

```
# 현재 사용자에게 영구 저장 (새 창부터 적용)
setx ANTHROPIC_API_KEY "sk-ant-..."
setx CLAUDE_MODEL "claude-sonnet-5-5"
# 지금 창에서만 쓸 때
$env:ANTHROPIC_API_KEY = "sk-ant-..."
```

macOS

터미널 (zsh)

```
echo 'export ANTHROPIC_API_KEY="sk-ant-..."' >> ~/.zshrc
echo 'export CLAUDE_MODEL="claude-sonnet-5-5"' >> ~/.zshrc
source ~/.zshrc
```

- `.env` 파일을 쓰면 `.gitignore` 에 꼭 넣습니다. 키가 담긴 파일을 저장소에 올리지 않습니다.
- OpenAI 비교 실습(선택)은 `OPENAI_API_KEY`, Oracle 실습은 `ORA_USER` · `ORA_PASSWORD` · `ORA_DSN` 을 같은 방법으로 지정합니다.

사내 폐쇄망으로 옮기기

반입은 회사의 자료 반입 절차와 보안 검토를 따릅니다. 아래는 인터넷 PC에서 미리 받아 두는 기술적인 방법입니다.

Python 패키지 (wheel 묶음)

pip

```
# 인터넷 PC: 사내 PC와 같은 OS · Python 버전으로 내려받기
pip download -r requirements.txt -d wheels --only-binary=:all: \
    --platform win_amd64 --python-version 3.13
# 사내 PC: 인터넷 없이 설치
pip install --no-index --find-links wheels -r requirements.txt
```

사내에 Nexus · Artifactory 같은 PyPI 미러가 있으면 `pip config set global.index-url https://사내미러/simple` 로 지정하는 편이 관리가 쉽습니다.

Maven 의존성

settings.xml

```
<!-- ~/.m2/settings.xml : 사내 Maven 저장소를 mirror 로 지정 -->
<settings>
  <mirrors>
    <mirror>
      <id>company-nexus</id>
      <mirrorOf>*</mirrorOf>
      <url>https://nexus.사내도메인/repository/maven-public/</url>
    </mirror>
  </mirrors>
</settings>
```

Maven

```
# 사내 저장소가 없을 때: 인터넷 PC에서 의존성을 모두 받아 ~/.m2/repository 를 통째로 옮깁니다
./mvnw dependency:go-offline
# 사내 PC: 오프라인 모드로 빌드
./mvnw -o -q compile
```

Maven Wrapper 도 처음 실행할 때 Maven 배포본을 내려받습니다. 사내에서는 환경변수 `MVNW_REPOURL` 에 사내 저장소 주소를 지정하거나, `.mvn/wrapper/maven-wrapper.properties` 의 `distributionUrl`

을 바꿉니다. 인터넷 PC의 `~/.m2/wrapper` 폴더를 그대로 옮겨도 됩니다.

모델 · 컨테이너 이미지

bash

```
# Docker 이미지 반입
docker pull pgvector/pgvector:pg17
docker save -o pgvector.tar pgvector/pgvector:pg17
docker load -i pgvector.tar          # 사내 PC
# 임베딩 모델(Hugging Face)은 인터넷 PC에서 한 번 실행해 캐시를 만든 뒤
# ~/.cache/huggingface 폴더를 옮기고 HF_HUB_OFFLINE=1 로 실행합니다
```

설치 점검

아래 스크립트는 버전과 키 설정 여부만 확인하고, 외부로 요청을 보내지 않습니다. 저장소의 `labs/setup/` 폴더에 있습니다.

labs/setup/check_env.py

```
"""실습 환경 점검 (Python 트랙)

    python labs/setup/check_env.py

설치한 도구의 버전과 API 키 설정 여부를 한 번에 보여 줍니다. 네트워크 호출은 하지 않습니다.
"""
import os
import platform
import shutil
import subprocess
import sys


def ver(cmd):
    exe = shutil.which(cmd[0])
    if not exe:
        return None
    try:
        out = subprocess.run(cmd, capture_output=True, text=True, timeout=15)
        return (out.stdout or out.stderr).strip().splitlines()[0]
    except Exception as e:  # 실행은 되지만 버전 출력이 이상한 경우
        return f"(확인 실패: {e})"


def row(name, value, need):
    mark = "OK " if value else ("-- " if not need else "없음")
```

```
    print(f"[{mark}] {name:<22} {value or ('선택' if not need else '설치 필요')}")


print(f"OS: {platform.system()} {platform.release()} / Python {platform.python_version()}
ok = sys.version_info >= (3, 11)
row("Python 3.11 이상", platform.python_version() if ok else None, True)
row("가상환경(venv)", sys.prefix if sys.prefix != sys.base_prefix else None, False)
row("pip", ver([sys.executable, "-m", "pip", "--version"]), True)
```

labs/setup/java/CheckEnv.java

```
// 실습 환경 점검 (Java 트랙) — 의존성 없이 단일 파일로 실행합니다.
//   java labs/setup/java/CheckEnv.java
// JDK 버전, JAVA_HOME, Maven Wrapper, API 키 설정 여부를 보여 줍니다. 네트워크 호출은 하지 않습
import java.nio.file.*;
import java.util.*;

public class CheckEnv {
    static void row(String name, String value, boolean need) {
        String mark = value != null ? "OK " : (need ? "없음" : "-- ");
        System.out.printf("[%s] %-22s %s%n", mark, name, value != null ? value : (need ?
    }

    static String run(String... cmd) {
        try {
            Process p = new ProcessBuilder(cmd).redirectErrorStream(true).start();
            String out = new String(p.getInputStream().readAllBytes()).strip();
            p.waitFor();
            return out.isEmpty() ? null : out.lines().findFirst().orElse(null);
        } catch (Exception e) {
            return null;
        }
    }

    public static void main(String[] args) {
        boolean win = System.getProperty("os.name").toLowerCase().contains("win");
        int feature = Runtime.version().feature();
        System.out.println("OS: " + System.getProperty("os.name") + " / JDK " + Runtime.
                + " (" + System.getProperty("java.vendor") + ")");
        row("JDK 21 이상", feature >= 21 ? String.valueOf(Runtime.version()) : null, true
        row("JAVA_HOME", System.getenv("JAVA_HOME"), false);
        Path wrapper = Path.of(win ? "mvnw.cmd" : "mvnw");
        row("Maven Wrapper(현재 폴더)", Files.exists(wrapper) ? wrapper.toAbsolutePath().
        row("Maven(전역)", run(win ? "mvn.cmd" : "mvn", "-v"), false);
        row("Git", run("git", "--version"), true);
        row("Python", run(win ? "python" : "python3", "--version"), false);
```

실행

```
python labs/setup/check_env.py
java labs/setup/java/CheckEnv.java     # JDK 11 이상은 단일 파일을 바로 실행할 수 있습니다
```

자주 막히는 문제

증상	원인	해결
`Activate.ps1` 실행이 막힘	PowerShell 실행 정책	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` 한 번 실행
`python` 을 치면 스토어가 열림	Windows 앱 실행 별칭	`py -3.13` 으로 실행하거나 설정 → 앱 실행 별칭에서 python 끄기
pip · Maven 에서 SSL 인증서 오류	회사 프록시가 인증서를 바꿔 끼움	사내 루트 인증서를 받아 pip 는 `pip config set global.cert 경로.pem`, Java 는 `keytool -importcert -cacerts -file 경로.cer -alias company`
`java -version` 이 예전 버전	PATH 에 다른 JDK 가 먼저 있음	`JAVA_HOME` 을 Temurin 21 로 바꾸고 PATH 의 `%JAVA_HOME%\bin` 을 맨 앞으로
모델 다운로드 · 빌드 경로 오류	Windows 사용자 폴더 이름이 한글	실습 폴더를 `C:\study` 처럼 영문 경로에 두고, `HF_HOME` 도 영문 경로로 지정
포트 사용 중 (8080, 5678, 5432)	다른 프로그램이 같은 포트 사용	Spring 은 `--server.port=8081`, Docker 는 `-p 15432:5432` 처럼 바꿔서 실행